

**Course: Fundamentals of Generative AI and Large Language
Models: Theory and Practice**

**Week 3: Variational Autoencoders (VAEs):
Foundations**

Today's Session Overview

- Decoder part of VAE
- Evidence Lower Bound (ELBO)
- Loss function of VAE

Variational Autoencoders (VAEs): Decoder

Once the encoder gives $\mu_\phi(x)$ and $\sigma_\phi^2(x)$ of the approximate posterior distribution,

$$q_\phi(z | x) = \mathcal{N}(\mu_\phi(x), \sigma_\phi^2(x))$$

The latent vector z is sampled from this distribution:

$$z \sim \mathcal{N}(\mu_\phi(x), \sigma_\phi^2(x))$$

The sampled latent vector z is then passed to the decoder.

The role of the decoder is to generate or reconstruct data from the latent vector (z).

The decoder is a neural network that takes (z) as input and outputs the parameters of the likelihood distribution:

$$p_\theta(x | z)$$

The type of parameters produced by the decoder depends on the type of data being generated.

For continuous or real-valued data,

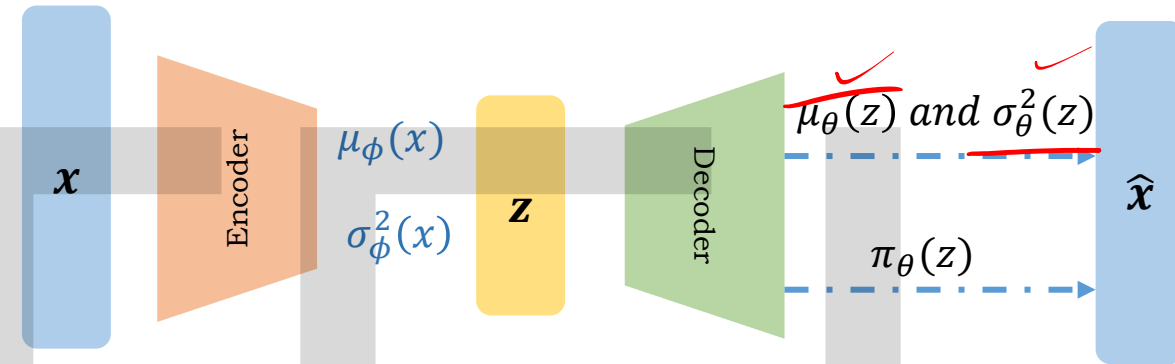
$$z \rightarrow \text{Decoder} \rightarrow \mu_\theta(z), \sigma_\theta^2(z) \rightarrow \hat{x}$$

These two values define a Gaussian distribution:

$$p_\theta(x | z) = \mathcal{N}(\mu_\theta(z), \sigma_\theta^2(z))$$

Then, probabilistically the output can be generated by sampling:

$$\hat{x} \sim \mathcal{N}(\mu_\theta(z), \sigma_\theta^2(z))$$



For binary-valued data, the decoder outputs a probability for each feature or pixel

$$z \rightarrow \text{Decoder} \rightarrow \pi_\theta(z) \rightarrow \hat{x}$$

This defines a Bernoulli distribution:

$$p_\theta(x | z) = \text{Bernoulli}(\pi_\theta(z))$$

Then, probabilistically the output can be generated by sampling:

$$\hat{x} \sim \text{Bernoulli}(\pi_\theta(z))$$

This means the generated output $\hat{x}$ is randomly drawn from that Gaussian distribution.

Variational Autoencoders (VAEs): Decoder

μ_1, σ_1^2 μ_2, σ_2^2 $\mathbf{x} = [0.80, 0.10, 0.60, 0.30, 0.90]$

	z_1	z_2
z^1		
z^2		
z^3		
$\vdots$		
z^n		

For one input $\mathbf{x}$, there is not only one possible $\mathbf{z}$.
There can be many possible $\mathbf{z}$ values that can represent the same input.

$z^1, z^2, z^3, \dots$ are sampled from the same distribution $q_\phi(\mathbf{z} | \mathbf{x})$

Sample 1: $z^1 = [0.56, -0.11]$, **Sample 2:** $z^2 = [0.43, -0.32]$

Sample 3: $z^3 = [0.61, -0.24]$

Each sampled $\mathbf{z}$ is passed to the decoder.

The decoder then tries to reconstruct or generate the data:

$$\mathbf{z} \rightarrow \text{Decoder} \rightarrow \hat{\mathbf{x}}$$

So, for the same input $\mathbf{x}$:

$$z^1 \rightarrow \text{Decoder} \rightarrow \hat{\mathbf{x}}^1 = [0.79, 0.12, 0.58, 0.32, 0.88]$$

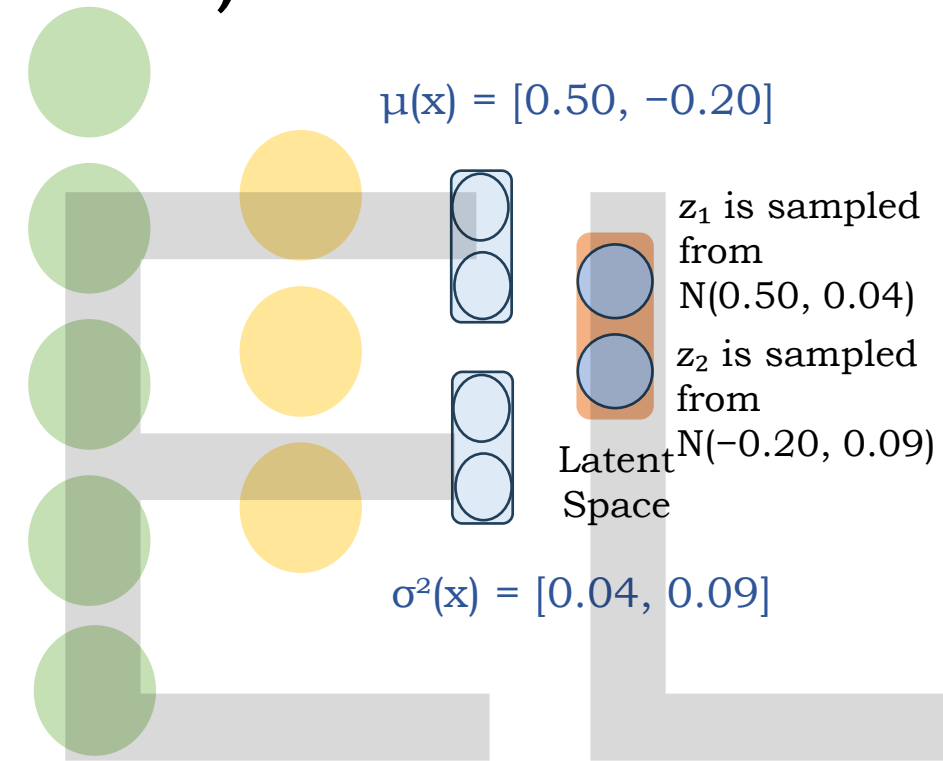
$$z^2 \rightarrow \text{Decoder} \rightarrow \hat{\mathbf{x}}^2 = [0.76, 0.14, 0.61, 0.28, 0.86]$$

$$z^3 \rightarrow \text{Decoder} \rightarrow \hat{\mathbf{x}}^3 = [0.82, 0.09, 0.57, 0.34, 0.91]$$

The reconstructed outputs are slightly different because different latent vectors were sampled. However, all of them remain similar to the original input: $\mathbf{x} = [0.80, 0.10, 0.60, 0.30, 0.90]$

There are infinitely many possible $\mathbf{z}$'s

Therefore, one observed input $\mathbf{x}$ can correspond to many possible latent representations $\mathbf{z}$.



Now the reverse question is:

Given this real training sample $\mathbf{x}$, which latent representation $\mathbf{z}$ produce this sample through the decoder?

Variational Autoencoders (VAEs): Decoder

We want to know which latent variable z could have generated the observed input x . According to Bayes' rule, this requires computing:

Using Bayes' rule:

$$p(z | x) = \frac{p(x | z) p(z)}{p(x)}$$

The denominator is:

$$p(x) = \int p(x | z) p(z) dz$$

The difficulty is that this integral requires considering **all possible latent variables z** that could have generated the observed data point x .

For neural network decoders, **this integral is usually intractable (difficult).**

Here the prior is: $p(z) = \mathcal{N}(0, I)$

For each possible z , the decoder gives the probability that this z could generate the observed input x :

$$p(x | z^1), p(x | z^2), p(x | z^3) \text{ and so on.}$$

But each latent vector also has a prior probability:

$$p(z^1), p(z^2), p(z^3), \dots$$

Latent vectors near the centre of the latent space, around zero, have higher density, while vectors far away from zero have lower density.

For example:

$$z^1 = [0.2, -0.1]$$

is close to the origin, so it has relatively high prior density. Whereas:

$$z^2 = [4.0, -3.5]$$

is far from the origin, so it has very low prior density.

Therefore, the contribution of each possible latent vector is: $p(x | z^1)p(z^1), p(x | z^2)p(z^2), p(x | z^3)p(z^3)$ and so on.

Variational Autoencoders (VAEs): Evidence Lower Bound (ELBO)

Suppose the training data contains many images of handwritten digits. After training, the model should give high probability to digit-like images and low probability to random noise images.

Ideally, we want to maximize:

$$\log p_{\theta}(x)$$

Computing $p_{\theta}(x)$ is difficult because: requires checking all possible latent variables z that could have generated x .

So, directly maximizing $\log p_{\theta}(x)$ is usually not possible. Therefore, VAE maximizes a simpler objective called:

Evidence Lower Bound (ELBO)

ELBO is a computable lower bound of $\log p_{\theta}(x)$

So instead of maximizing: $\log p_{\theta}(x)$

VAE maximizes: **ELBO**

Variational Autoencoders (VAEs): Evidence Lower Bound (ELBO)

The ELBO is:

$$\mathcal{L}_{ELBO}(x) = \mathbb{E}_{q_{\phi}(z|x)}[\log p_{\theta}(x|z)] - D_{KL}(q_{\phi}(z|x) \parallel p(z))$$

Reconstruction

KL regularization term

If the encoder samples z from $q_{\phi}(z|x)$ can the decoder reconstruct the original input x with high probability?

Is the encoder distribution $q_{\phi}(z|x)$ close to the prior distribution $p(z)$?

The first term in the VAE Loss is log-likelihood. Always we expect the model to assign high probability to the observed data.

$$i.e. \max \mathbb{E}_{q_{\phi}(z|x)}[\log p_{\theta}(x|z)]$$

Neural networks are trained to minimize the loss not maximize. So, we convert max into minimum.

$$\max(f) \Leftrightarrow \min(-f)$$

So:

$$\mathcal{L}_{VAE}(x) = -\mathbb{E}_{q_{\phi}(z|x)}[\log p_{\theta}(x|z)] + D_{KL}(q_{\phi}(z|x) \parallel p(z))$$

That is:

$$\mathcal{L}_{VAE} = \text{Reconstruction Loss} + \text{KL Divergence Loss}$$

Variational Autoencoders (VAEs): Loss Function

$$L(x) = \underbrace{-\mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x|z)]}_{\text{Reconstruction}} + \underbrace{KL(q_\phi(z|x) \parallel p(z))}_{\text{Regularization}}$$

Encourages the encoder distribution $q_\phi(z|x)$ to remain close to the prior distribution $p(z)$.

Reconstruction



Mean squared error

The latent vector z is sampled from:

$$z \sim \underline{q_\phi(z|x)} = \mathcal{N}(\mu(x), \sigma^2(x))$$

Since sampling is a random operation, Randomness breaks gradient flow, making backpropagation impossible

Regularization



$$L = L_{KL}(\mathcal{N}(\mu, \sigma^2) \parallel \mathcal{N}(0,1))$$
$$L_{KL} = \frac{1}{2}[\mu^2 + \sigma^2 - \log \sigma^2 - 1]$$

When both distributions are Gaussian, the KL divergence has a closed-form expression

So, the question is: **How do we compute gradients when part of the network involves sampling?**

This motivates the Reparameterization Trick

Next Session

➤ Reparameterization Trick

NPTREL

NIPTELL

Thank you